Práctica 1 — Catálogo de Recursos

Estructura de Datos y Algoritmia — Curso 2026-2027
Lenguaje: C++ (estándar C++11)
Entrega: **25/09/2026 23:59** en el servidor de prácticas https://pracdlsi.dlsi.ua.es
La evaluación se hará mediante un examen práctico en ordenador (ver Sección 13).

---

1. Objetivo

Familiarizarse con la **programación orientada a objetos en C++** y con el **diseño de Tipos Abstractos de Datos (TAD)**.

Se trabajará con:

- Definición e implementación de **clases en C++**
- **Encapsulación de datos**
- Implementación de la **forma canónica de una clase**
- Uso de **vectores (`vector`)**
- Diseño de **operaciones de un TAD**
- Compilación, ejecución y **depuración** de programas en C++
- El **autocorrector** de la asignatura (ver el tutorial)

El resultado será un pequeño sistema para gestionar un **catálogo de recursos**.

---

2. Descripción del problema

Se quiere implementar un programa que gestione un **catálogo de recursos**.

Cada recurso tiene:

- Un **identificador** único (`id`)
- Un **título** (`title`)
- Un **año** (`year`)
- Un conjunto de **etiquetas** (`tags`)

El catálogo permite:

- Añadir recursos
- Eliminarlos
- Buscarlos por identificador
- Filtrarlos por etiqueta
- Cargar la información desde un fichero

---

3. Clase `Resource`

3.1 Atributos (privados)

Atributo	Tipo	Restricciones
`id`	`string`	Identificador; vacío únicamente en el recurso construido por defecto
`title`	`string`	Puede contener espacios
`year`	`int`	Entero
`tags`	`vector<string>`	Minúsculas, sin duplicados, orden lexicográfico

3.2 Forma canónica

```
Resource();
Resource(const string& id, int year, const string& title);
Resource(const Resource& other);
Resource& operator=(const Resource& other);
~Resource();
```

El constructor por defecto inicializa `id` y `title` a la cadena vacía y `year` a 0.

3.3 Getters

```
string getId() const;
string getTitle() const;
int getYear() const;
vector<string> getTags() const;
```

3.4 Gestión de etiquetas

```
bool addTag(const string& tag);
```

- Las etiquetas sólo pueden contener caracteres ASCII A-Z, a-z, 0-9.
- Elimina cualquier espacio al principio y al final de la etiqueta.
- Convierte la etiqueta a **minúsculas**.
- No añade duplicados ni etiquetas vacías.
- Mantiene el vector **ordenado lexicográficamente**.
- Devuelve `true` si se añade, `false` en caso contrario.

```
bool removeTag(const string& tag);
```

- Normaliza la etiqueta a minúsculas antes de buscar, sin eliminar espacios al principio o al final.
- No hace ninguna otra transformación sobre la etiqueta de entrada.
- Devuelve `true` si se elimina, `false` si no existía.

```
bool hasTag(const string& tag) const;
```

- Normaliza la etiqueta a minúsculas antes de buscar, sin eliminar espacios al principio o al final.

- No hace ninguna otra transformación sobre la etiqueta de entrada.
- Devuelve `true` si el recurso contiene la etiqueta.

3.5 Operadores

```
bool operator==(const Resource& other) const;
```

Dos recursos son iguales si tienen el **mismo `id`**.

```
bool operator<(const Resource& other) const;
```

Compara por `id` (orden lexicográfico).

```
friend ostream& operator<<(ostream& os, const Resource& r);
```

Formato exacto (una sola línea, sin espacios extra, **no debe imprimir un salto de línea al final**):

```
<id> (<year>) <title> [tag1,tag2,tag3]
```

- Las tags van separadas por comas **sin espacios**.
- Si no hay tags: `[]`.

Ejemplo:

```
bk01 (1999) Clean Code [craft,software]
bk02 (2020) Empty Tags []
bk03 (2020) Modern C++ [cpp]
```

---

4. Clase TAD `Catalog`

4.1 Atributos

```
vector<Resource> data
```

No puede haber **dos recursos con el mismo `id`**.

4.2 Forma canónica

```
Catalog();
Catalog(const Catalog& other);
Catalog& operator=(const Catalog& other);
~Catalog();
```

4.3 Métodos

```
bool add(const Resource& r);
```

Añade el recurso si su `id` no está vacío y no existe otro igual (de acuerdo al comportamiento del operador `==`). Devuelve `true` si se añade, `false` en caso contrario.

```
bool removeById(const string& id);
```

Elimina el recurso con ese identificador. Devuelve `true` si se elimina, `false` si no existe.

```
Resource findById(const string& id) const;
```

Devuelve el recurso si existe, o un nuevo recurso vacío (creado con el constructor por defecto) en caso contrario.

```
vector<Resource> findByTag(const string& tag) const;
```

- Devuelve los recursos que contienen esa etiqueta, ordenados según la relación de orden que establece el `operator<` de `Resource`.
- Las reglas para comparar dos etiquetas están definidas en `Resource::hasTag`.

```
int size() const;
```

Devuelve el número de recursos almacenados.

```
bool loadFromFile(const string& filename);
```

- Abre el fichero. Si falla: devuelve `false`.
- Si abre correctamente: **borra todos los datos actuales** del catálogo.
- Lee línea a línea y añade los recursos válidos.
- Si hay recursos con `id` repetido sólo se guarda el primero.
- Devuelve `true`.

---

5. Formato de fichero

Cada línea representa un recurso, con 4 campos separados por `|`:

```
id|year|title|tag1,tag2,tag3
```

Si no hay etiquetas, el último campo queda vacío:

```
id|year|title|
```

Reglas de validación (mínimas)

- Si la línea no tiene exactamente **3 separadores**: se ignora el recurso.
- Si `id` está **vacío**: se ignora el recurso.
- Las tags se separan por **comas**.
- Las `tags` se añaden usando `Resource::addTag` para forzar las mismas reglas de validación.
- No es necesario validar `year`, puedes asumir que será un entero válido.

Ejemplo de fichero (`data/sample_ok.txt`)

```
bk01|1999|Clean Code|Software,CRAFT
bk02|2008|Introduction to Algorithms|algorithms,CS,Algorithms
bk03|2015|Design Patterns|patterns,OOP,design
```

```
bk04|1994|The Pragmatic Programmer|craft,Software
bk05|2011|Structure and Interpretation|cs,LISP,programming
bk06|2020|Modern C++|cpp,Programming
```

---

6. Aplicación `main.cc`

El programa lee comandos desde **stdin** y escribe en **stdout**. No debe mostrar prompts ni mensajes adicionales.

En las pruebas no se pasará nunca argumentos de más, no es necesario comprobarlo.

Comandos

`LOAD <file>` Carga un catálogo desde fichero (borra datos anteriores).

- Éxito: `OK <n>` (donde `n` = número de recursos cargados)
- Error al abrir: `ERROR`

`ADD <id> <year> <title> <tags>`

- El título usa **guiones bajos** (`_`) en lugar de espacios, debes reemplazarlos por espacios para crear el recurso.
- Las tags son `tag1,tag2,...` o `-` si no hay tags.

Salidas: - `OK` si se añade correctamente. - `DUPLICATE` si ya existe un recurso con ese `id`.

Ejemplo: `ADD bk01 1999 Clean_Code software,craft`

`DEL <id>`

- `OK` si se elimina.
- `NOTFOUND` si no existe.

`GET <id>`

- Imprime el recurso (formato `operator<<`).
- `NOTFOUND` si no existe.

`FINDTAG <tag>`

- Imprime `COUNT <k>` seguido de `k` líneas con los recursos encontrados (ordenados por `id` en orden lexicográfico ascendente).

`EXIT` Termina el programa sin imprimir nada.

Errores Si un comando no se reconoce o **faltan argumentos**, se imprime: `INVALIDCMD`

---

7. Ficheros a implementar

En `src/` se proporcionan **plantillas** con la estructura mínima de cada fichero. Debes completar:

```
src/
  Resource.h
  Resource.cc
  Catalog.h
  Catalog.cc
  main.cc
```

Restricciones de implementación:

- No puedes usar variables globales.
- No puedes modificar las signaturas de los métodos descritos en este enunciado (puedes añadir métodos privados auxiliares).
- El código debe compilar **sin errores ni warnings** con `g++ -std=c++11 -Wall -Wextra`.

---

8. Compilación

```
make            # compila src/ -> ejecutable 'catalog'
make clean      # limpia binarios
```

El programa se compila con C++11 y los flags `-Wall -Wextra -g`.

---

9. Datos de prueba

Se proporcionan ficheros en `data/`:

Fichero	Contenido
`sample_ok.txt`	6 recursos válidos (con tags duplicadas y mayúsculas)
`sample_mixed.txt`	Mezcla de líneas válidas e inválidas (3 válidas)
`sample_empty.txt`	Fichero vacío (0 recursos)

---

10. Pruebas automáticas

La práctica completa se evalúa con **pruebas automáticas** organizadas en 3 grupos:

Grupo	Descripción
A	Clase `Resource`
B	Clase `Catalog`
C	Aplicación CLI `catalog`

Las pruebas proporcionadas cubren gran parte de los casos posibles, pero es posible que haya algunos **casos límite** que no se estén probando. Todos esos casos están descritos en este enunciado.

Debes asegurarte de que tu código cumple al 100% con las especificaciones creando tus propias pruebas del grupo Z.

Consulta el **tutorial** para aprender a ejecutar las pruebas, interpretar los resultados y depurar errores:

```
./run_tests.sh          # todas las pruebas
./run_tests.sh A        # solo el grupo A
./run_tests.sh A01      # una prueba concreta
```

---

11. Criterios de corrección

- Una prueba solo puntúa si **pasa** (PASS). Los resultados FAIL, ERROR, CRASH, MEMORY y TIMEOUT no puntúan.
- El código que **no compila** no puntúa en las pruebas afectadas: comprueba siempre que `make` termina sin errores ni warnings.
- Se penalizará especialmente: no respetar los formatos de salida, modificar las signaturas indicadas y el uso de variables globales.

---

12. Entrega

Antes de entregar debes modificar el fichero `AUTHOR.txt` y añadir tu nombre completo y DNI.

Para preparar la entrega ejecuta el script `entrega.sh` en la carpeta de la práctica para generar el fichero `p1_entrega.tgz` que debes subir al servidor de prácticas:

```
./entrega.sh
# Este script creará el fichero entregable p1_entrega.tgz
```

---

13. Examen

En el examen práctico (sin Internet ni asistentes de IA) recibirás:

1. **Tu propia implementación** de esta práctica, tal y como la hayas preparado en casa.
2. Las **pruebas completas** de la práctica.
3. Un enunciado con una **ampliación o modificación** (por ejemplo: un nuevo método del TAD con su comando asociado) y sus **pruebas específicas**.

Si has escrito y comprendido tu práctica, la ampliación es asequible en el tiempo disponible. Si tu práctica está incompleta, el tiempo del examen difícilmente alcanzará para completarla e implementar la ampliación.

---

14. Errores frecuentes y recomendaciones

- **Normalización de tags:** recuerda recortar espacios y pasar a minúsculas **antes** de validar duplicados. Comprueba qué debe ocurrir con `" C++ "` (carácter no permitido) o `"  Craft  "`.
- **Orden lexicográfico:** `addTag` debe insertar manteniendo el orden; no basta con hacer `push_back`.
- **`operator<<`:** respeta el formato exacto (espacios, corchetes, comas). La mayoría de fallos del grupo C vienen de un `operator<<` incorrecto.
- **`loadFromFile`:** borra el contenido previo **solo** si el fichero se abre correctamente. Cuenta únicamente las líneas válidas.
- **Paso por referencia y `const`:** los métodos que no modifican el objeto deben ser `const`; los parámetros grandes se pasan por referencia constante. El autocorrector no compilará si faltan los `const`.
- Trabaja de forma **incremental**: primero `Resource` (grupo A), después `Catalog` (grupo B) y por último `main.cc` (grupo C).